12. ROS 2 Metapackage Introduction

1. Introduction to Metapackages

Completing a systematic function may involve multiple packages. For example, when implementing a robot navigation module, this module has different sub-level packages such as mapping, localization, and path planning... So when the caller installs this module, does it need to install every package one by one?

Obviously, installing packages one by one is inefficient. In ROS 2, you can bundle multiple packages into one. When installing a certain function module, you can directly call the packaged package. This package is also called a metapackage.

MetaPackage is a file management system concept in Linux. It is a virtual package in ROS 2. It has no substantive content inside, but it depends on other software packages. In this way, other packages can be combined together. We can regard it as the table of contents index of a book, telling us which sub-packages are in this package collection and where to download them.

For example: the `sudo apt install ros-${ROS_DISTRO}-desktop` command uses a metapackage when installing ROS 2. This metapackage depends on some other packages in ROS 2, and the dependencies will be installed together when installing this package.

Classic metapackage case: navigation2

- navigation2 GitHub repository: [GitHub - ros-navigation/navigation2: ROS 2 Navigation Framework and System]

2. Purpose

It facilitates user installation. We only need this one package to organize and install other related packages together.

3. Implementation

1. Create a new package

```
ros2 pkg create pkg_metapackage
```

2. Modify the package.xml file and add the packages that are depended on at runtime

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>pkg_metapackage</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="1461190907@qq.com">root</maintainer>
  <license>TODO: License declaration</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <exec_depend>pkg_interfaces</exec_depend>
```

```
  <exec_depend>pkg_helloworld_py</exec_depend>
  <exec_depend>pkg_topic</exec_depend>
  <exec_depend>pkg_service</exec_depend>
  <exec_depend>pkg_action</exec_depend>
  <exec_depend>pkg_param</exec_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

3. The content of the file CMakeLists.txt is as follows

```
cmake_minimum_required(VERSION 3.5)
project(pkg_metapackage)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)

ament_package()
```

4. Compile the metapackage

- There will be no actual executable files

```
colcon build --packages-select pkg_metapackage
```

